

This documentation describes the Python script pipeline developed to replicate the OUTPUT tab of the [referenced spreadsheet](#). The pipeline is designed to process and integrate various data sources to produce a comprehensive and structured analytical output Excel file.

```
├── input_data
│   ├── CENSUS_MHI_STATE.csv -> median house income data
│   ├── CENSUS_POPULATION_STATE.tsv -> population data
│   ├── REDFIN_MEDIAN_SALE_PRICE.csv -> median sale price data
│   ├── KEYS.csv -> region names mapping file
│   ├── key_row.csv -> file that contains regions of interest
│   └── override_data.csv -> optional file for manual hardcoded fields in OUTPUT.xlsx file
├── output
│   └── OUTPUT.xlsx -> generated output file
└── src
    ├── generate_output.py -> command line script that generates OUTPUT.xlsx file
    ├── utils.py -> utility functions used by main generate_output.py script
    └── visualization.ipynb -> visualizations script
```

This directory contains main provided 4 data sources that contain the data needed for generating `OUTPUT.xlsx` file in addition to two files which are:

- `key_row.csv`: This file contains the regions that will be retrieved in the `OUTPUT.xlsx` file which is displayed in the first column
- `override_data.csv`: This file contains optional feature for overriding or providing data for regions that is not present in main data sources, as observed in the `OUTPUT.xlsx` file, there are some values that are not mapped through formulas but are hardcoded values, to better organize this option this file is provided

[illegible]

output

This directory contains the final generated `OUTPUT.xlsx` file.

src

This directory contains the core scripts responsible for data processing, output generation, and visualization.

Scripts Description

This section outlines the assumptions and core functionalities embedded within the Python scripts used to generate the final Excel output. The workflow centers around aggregating census and housing data, computing rankings & blurbs, and formatting the results into an exportable Excel sheet.

utils.py

This script serves as a utility module, providing reusable functions for data transformation, formatting, and output generation. Each function parameter's are described in docstring of with the docstring, key functions are:

get_regions_mapping

Maps region identifiers from the key dataset (KEYS.csv) to standardized Zillow region names that are used to retrieve different metrics across datasets. It is worth noting that `key_row` field in the dataset may contain duplicate values. To mimic the behavior of Excel's `MATCH` function with `match_type=0` (which returns the first exact match), duplicates are removed while retaining the first occurrence. This is handled in pandas using the following line: `key_row_mapper.drop_duplicates(['key_row'], keep='first', inplace=True)`

get_census_stats

Extracts specific labeled metrics (e.g., estimate of population or median of household income) for all regions from census datasets.

This function is tailored to work with census data for population and median household income (MHI), as both share the same structural format.

add_census_stat_with_blurb

A higher-order function that wraps `get_census_stats`, with the following functionalities:

- Joining the requested metric to the main output DataFrame
- Generating rankings for each region
- Producing descriptive blurbs based on the metric values

Like `get_census_stats`, this function only works with population and MHI census data.

get_median_sale_price

Extracts and cleans the most recent median home sale price data from median sale price datasets.

`ordinal`

Converts numeric ranks into their ordinal string equivalents (e.g., 1 → 1st), and in case of empty rank (the ranked numerical column is null) returns empty string.

`get_rank`

Assigns ordinal rankings to numeric columns with optional ascending control.

`format_numbers`

Cleans and converts number strings (with symbols like \$ or ,) into Pandas numerical representation.

`generate_blurb`

Uses a provided template and placeholder mappings to generate human-readable summaries per region.

`load_override_data`

Loads optional override data (CSV format) for replacing or augmenting column values in the final output excel file.

`save_formatted_xlsx`

Saves the final DataFrame to an Excel file with formatted number and currency columns.

`generate_output.py`

This is the primary script responsible for orchestrating the full data pipeline. It performs the following steps:

1. Argument Parsing

The script accepts command-line arguments for input paths, file names, and other configurations. Default values are provided for all arguments to enable out-of-the-box usage. A detailed explanation of all available arguments is provided in the "How to Run" section.

2. Reading Data Sources

Reads all required input files.

3. Data Gathering

This stage begins population the output DataFrame by first, mapping the row_key regions to Zillow regions, then retrieve and generate the main 3 columns (main metric column, calculating rank, and generating the blurb) for census population, census MHI, median sale price datasets, and the calculated house affordability ratio metric.

4. Final Output

Cleans intermediate columns, apply formatting rules for numbers and currencies, and saves the final Excel file.

How to Run

Requirements

1. This project is built using **Python 3.10.11**. Please make sure to use this version (or a compatible one) to avoid compatibility issues.
2. Install requirements.txt from cmd with:

```
pip install -r requirements.txt
```

Running Script

The main script `generate_output.py` can be executed from the command line and accepts several configurable arguments to control input sources and output behavior.

Argument	Type	Default	Description
<code>--population-fname</code>	str	CENSUS_POPULATION_STATE.tsv	Filename of the census population data file inside <code>--data-dir</code> .
<code>--mhi-fname</code>	str	CENSUS_MHI_STATE.csv	Filename of the median household income CSV inside <code>--data-dir</code> .
<code>--msp-fname</code>	str	REDFIN_MEDIAN_SALE_PRICE.csv	Filename of the median sale price data file inside <code>--data-dir</code> .
<code>--region-keys-fname</code>	str	KEYS.csv	Filename of the region mapping keys file inside <code>--data-dir</code> .
<code>--override-data-fname</code>	str	(optional)	Filename of the override CSV file inside <code>--data-dir</code> .
<code>--data-dir</code>	str	input_data	Directory where the input files are located.
<code>--output-dir</code>	str	output	Directory where the output Excel file will be saved.
<code>--output-fname</code>	str	OUTPUT.xlsx	Name of the final generated Excel file.

Example

To run the script using default filenames and folder structure run the following on your command line from main project directory:

```
python ./src/generate_output.py
```

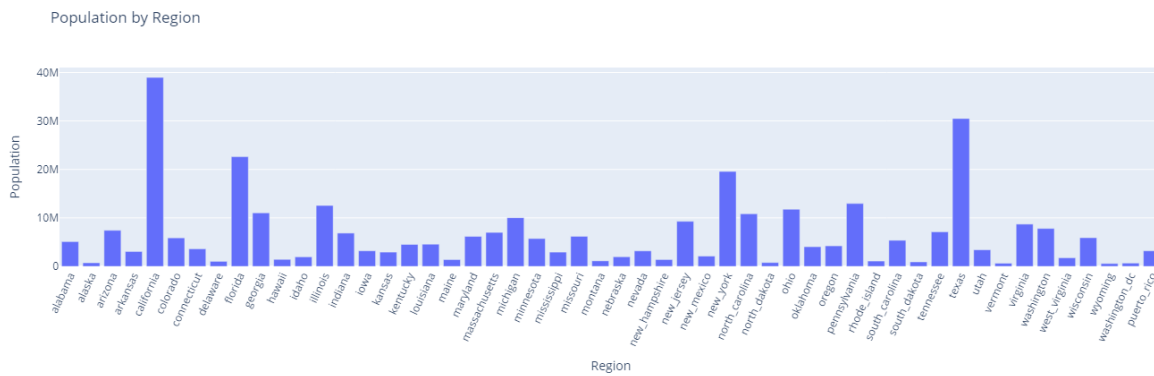
To generate identical Excel file as the referenced spreadsheet, use `--override-data-fname` option:

```
python ./src/generate_output.py --override-data-fname override_data.csv
```

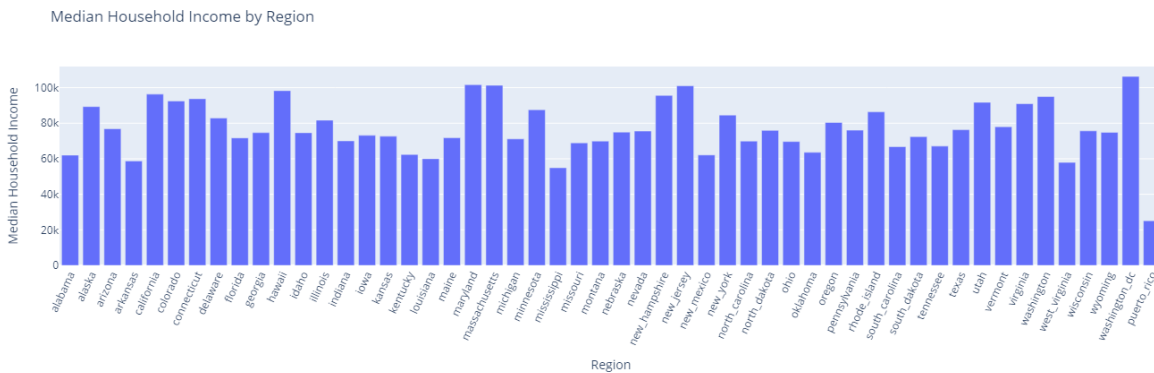
Bonus: Visualization

The following charts were created using Plotly to extract insights from the final output Excel file. For an interactive experience and higher resolution visuals, it's recommended to

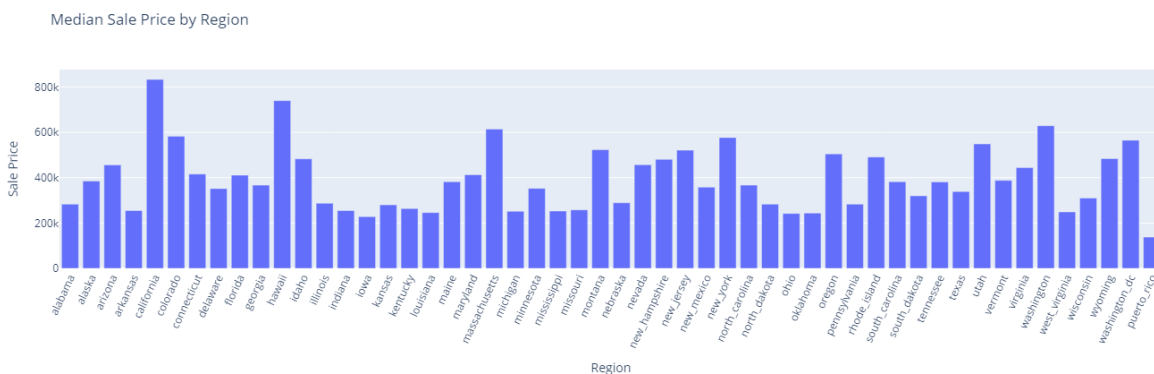
explore them through the Jupyter notebook located at `./src/visualization.ipynb`.



The plot highlights states like California, Florida, New York, and Texas are among the high-density regions. If these regions do not also have correspondingly high median household incomes, they tend to exhibit high home affordability ratio (indicating that housing is less affordable).

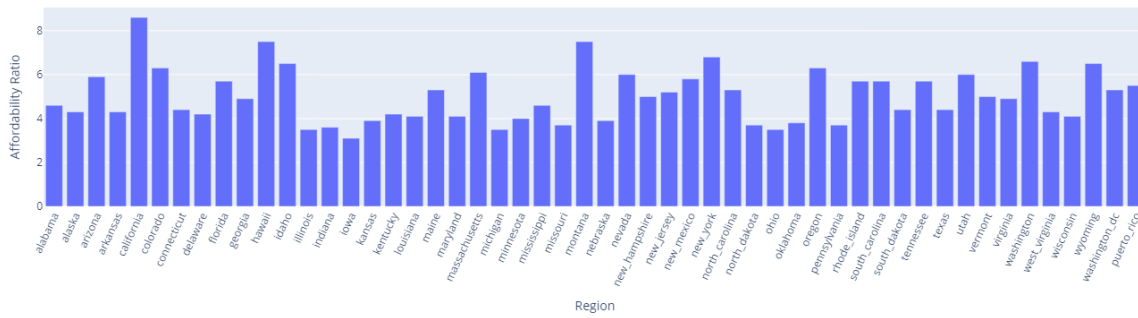


The plot indicates that no region stands out with exceptionally high median household income, while Puerto Rico notably has one of the lowest median household incomes among all regions.



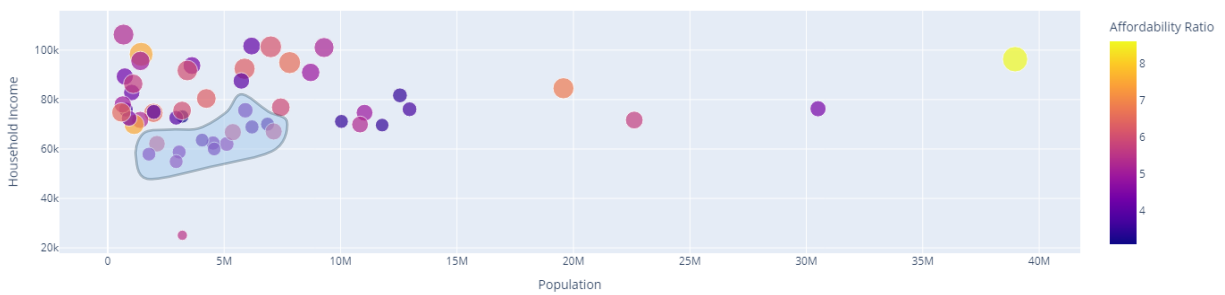
The plot shows that regions such as California, Hawaii, Massachusetts, and Washington have notably high median sale prices, with Hawaii standing out at the top after California. As observed in previous charts, Hawaii also has a low population, which is an interesting insight that may reflect the impact of high housing costs and geographic constraints on population density.

House Affordability Ratio by Region (Lower is More Affordable)



The chart indicates that regions such as California, Hawaii, Montana, New York, and Washington have high house affordability ratios, suggesting that homes in these areas are significantly less affordable relative to household income.

Population vs. Household Income (Bubble size = Home Price)



The chart reveals that there is a cluster of regions such as West Virginia, Mississippi, Arkansas, Oklahoma, Louisiana, Kentucky, Iowa, and Kansas tend to have both relatively low populations and lower household incomes, yet they achieve favorable housing affordability scores.